

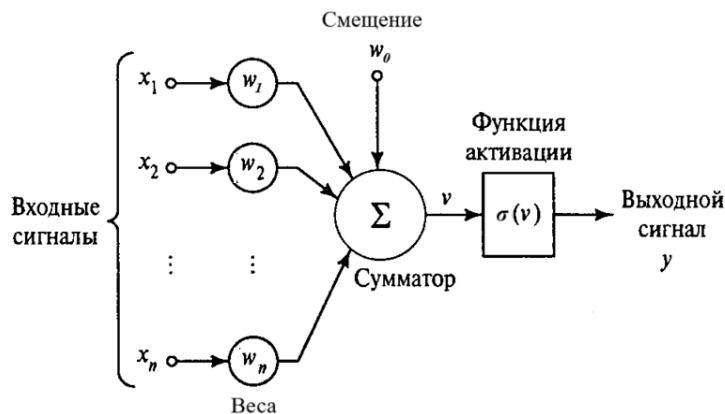
Лабораторная работа №5

“Нейронные сети. Многослойный перцептрон (MLP)”

Искусственные нейронные сети (ИНС) представляют собой математические модели, вдохновленные работой биологических нейронов. Они состоят из множества соединенных между собой элементов — **искусственных нейронов**, каждый из которых выполняет простые вычисления, но в совокупности сеть способна решать сложные задачи.

Искусственный нейрон

Искусственный нейрон — это базовый элемент искусственной нейронной сети, вдохновленный работой биологических нейронов в мозге человека. Схема одного нейрона следующая:



Искусственный нейрон, как и его биологический оригинал, имеют входные связи называемые **дендритами** и выходную связь — **аксон**. В искусственном нейроне каждый дендрит имеет свой вес. Для выражения коэффициента смещения добавляют дополнительную связь с весовым коэффициентом w_0 и входным значением x_0 , равным 1.

Попадая в нейрон, входные значения вектора x перемножаются с соответствующими весами и суммируются вместе со смещением:

$$v = \sum_{i=1}^n x_i \cdot w_i + w_0 = \sum_{i=0}^n x_i \cdot w_i, \text{ где } x_0 = 1$$

Далее эта сумма подается в основную часть, определяющую нейрон, в **функцию активации**, из которой получается результат работы нейрона на аксоне:

$$y = \sigma(v) = \sigma\left(\sum_{i=0}^n x_i \cdot w_i\right)$$

Аксон же может быть подсоединен к дендриту другого нейрона, тем самым передавая выходное значение одного нейрона на вход другого.

Функции активации

Нейрон можно сопоставить с *линейной моделью машинного обучения*, а функцию активации — с ее *гипотезой*. В зависимости от того, какая используется функция активации, результат вычисления можно сопоставить с поиском решения в виде линейной, сигмоидальной или другой функции.

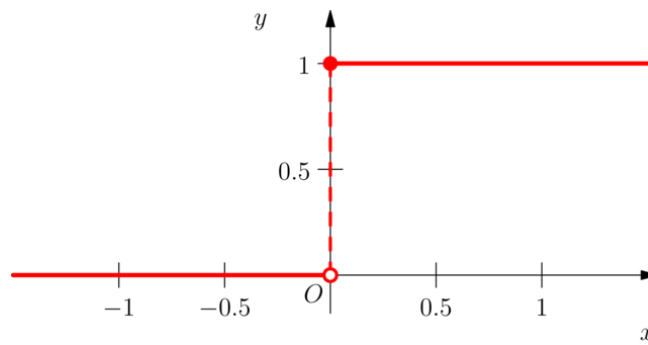
Существует немало функций активации. Функции активации сравниваются по следующим перечисленным параметрам, оказывающим различное влияние на итоговый результат:

- Нелинейность
- Непрерывная дифференцируемость
- Область значений
- Монотонность
- Гладкость
- Линейность в окрестности нуля

Рассмотрим наиболее распространенные виды функций активации и их основные свойства.

Одним из первых предложенных вариантов искусственного нейрона был **персептрон**. Он имеет **пороговую функцию активации**

$$\sigma(v) = [v > 0] = \left[\sum_{i=0}^n x_i \cdot w_i > 0 \right]$$



- Нелинейная;
- разрывная – порядок гладкости C^{-1} , производная не определена для $x = 0$;
- область значений: $\{0; 1\}$;
- монотонна;
- нелинейна в окрестности нуля.

Иногда используется в машинном обучении для задач классификации, поскольку выходное значение может быть либо 0, либо 1, что соответствует простейшей бинарной классификации. Тем не менее из-за того, что в окрестности нуля функция терпит разрыв первого рода, ее использование при обучении может быть затруднительно.

Линейная функция активации

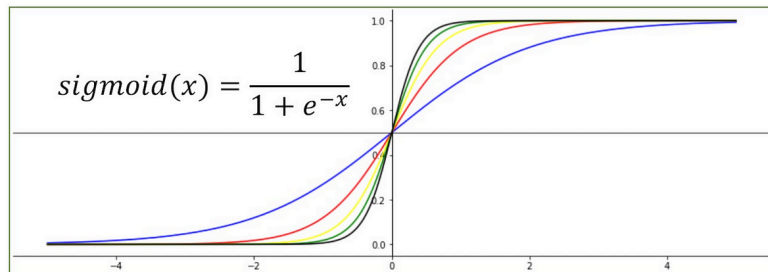
$$\sigma(v) = v = \sum_{i=0}^n x_i \cdot w_i$$

- Не обладает нелинейностью;
- непрерывно дифференцируема, порядок гладкости C^{∞} ;
- область значений: $(-\infty; \infty)$;
- монотонна;
- линейна в окрестности нуля.

По сути является просто сумматором. Использование этой функции активации позволяет смоделировать только линейную комбинацию признаков на выходе, т. е. нейронная сеть с линейной функцией активации способна осуществлять только линейную регрессию, из-за чего на практике линейная функция почти никогда не используется.

Логистическая сигмоидальная функция

$$\sigma(v) = \frac{1}{1 + e^{-v}}$$

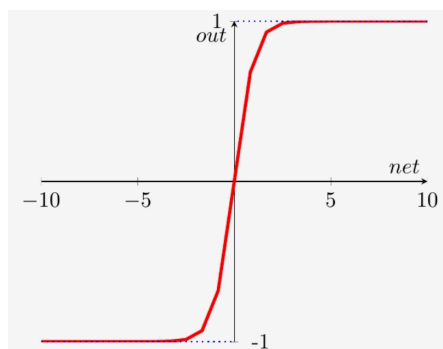


- нелинейная;
- непрерывно дифференцируема, порядок гладкости C^∞ ;
- область значений: $(0; 1)$;
- монотонна;
- нелинейна в окрестности нуля.

Логистическая сигмоидальная функция является одной из наиболее популярных функций активации для задач классификации, поскольку непрерывное выходное значение этой функции в диапазоне от 0 до 1 может быть интерпретировано как степень принадлежности к 1-му классу бинарной классификации. Помимо этого для некоторых других задач может быть полезным свойство этой функции ограничивать выходное значение в диапазоне $(0; 1)$.

Гиперболический тангенс

$$\sigma(v) = th\ v = \frac{e^v - e^{-v}}{e^v + e^{-v}}$$



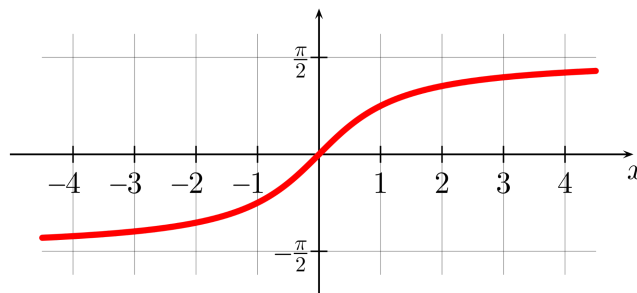
- нелинейная;
- непрерывно дифференцируема, порядок гладкости C^∞ ;

- область значений: $(-1; 1)$;
- монотонна;
- линейна в окрестности нуля.

Гиперболический тангенс также относится к сигмоидальным функциям; в отличие от логистической функции получаемое значение заключено в диапазон от -1 до 1, функция пересекает начало координат. Кроме того, в окрестности 0 функция ведет себя линейно. По сравнению с логистической функцией, промасштабированной до диапазона $(-1; 1)$, гиперболический тангенс не такой «плавный», т. е. смена угла между асимптотой и линейной функцией в окрестности нуля происходит более резко.

Арктангенс

$$\sigma(v) = \arctg v$$

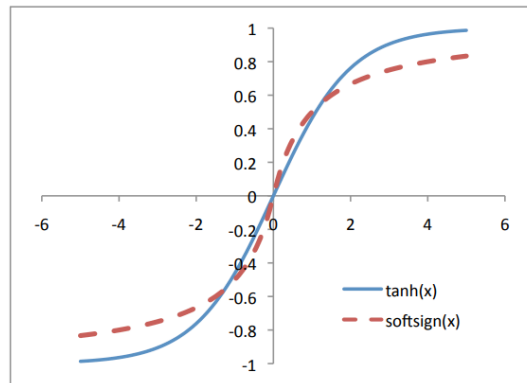


- нелинейная;
- непрерывно дифференцируема, порядок гладкости C^∞ ;
- область значений: $\left(-\frac{\pi}{2}; \frac{\pi}{2}\right)$;
- монотонна;
- линейна в окрестности нуля.

Еще один представитель сигмоидальных функций с более «плавным» переходом по сравнению с гиперболическим тангенсом и логистической функцией при масштабировании на одинаковый диапазон выходных значений.

Softsing

$$\sigma(v) = \frac{v}{1 + v}$$

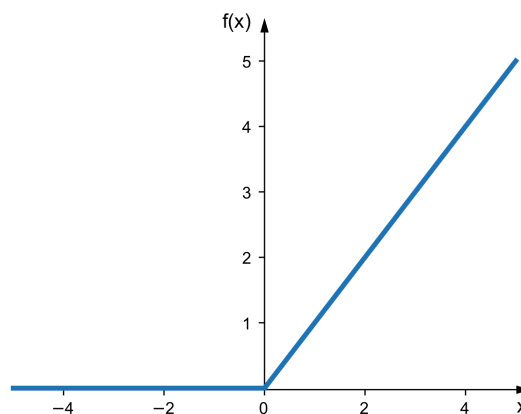


- нелинейная;
- непрерывно дифференцируема, порядок гладкости C^1 ;
- область значений: $(-1; 1)$;
- монотонна;
- линейна в окрестности нуля.

Softsign также относится к сигмоидальным функциям. Это наиболее плавная сигмоидальная кривая. Особенность этой функции – наличие разрыва второй производной в точке $x = 0$, что ограничивает ее применимость с ньютоновскими методами оптимизации.

Rectified Linear Unit (ReLU)

$$\sigma(v) = \max(0, v)$$



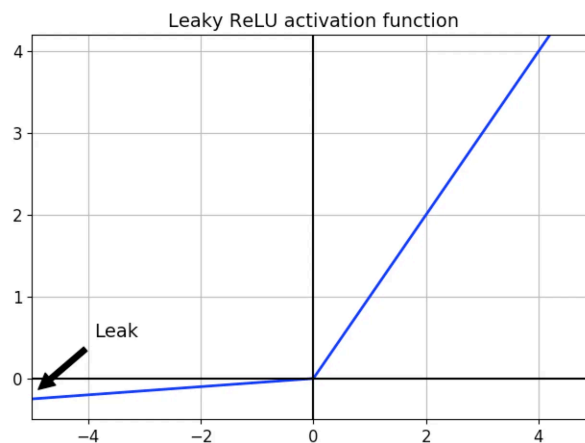
- нелинейная;
- непрерывная, разрыв производной в точке $x = 0$, порядок гладкости C^0 ;
- область значений: $[0; +\infty)$;
- монотонна;
- нелинейна в окрестности нуля.

ReLU – одна из наиболее популярных нелинейных функций активации. Эта функция активации получила широкое распространение в первую очередь из-за вычислительной простоты – по сути, для ее реализации необходимо единственное сравнение с нулем и не требуется более сложных арифметических операций. Кроме того, согласно ряду исследований такая функция активации обеспечивает подобный сигмоидальным функциям уровень нелинейности, при этом не ограничивая диапазон выходных значений, что делает ее крайне эффективной в глубоком обучении. Чтобы избавиться от проблемы несуществующей производной в окрестности нуля, при численном дифференцировании она искусственно принимается равной 0 или 1. Хотя ReLU нелинейна в окрестности нуля, на практике случайная инициализация весов может проводиться без ограничений, поскольку в таком случае в среднем у половины нейронов активации будут нулевые.

Параметрическая ReLU, ReLU с утечкой (Leaky) и стохастическая ReLU

$$y(x) = \begin{cases} x, & \text{если } x \geq 0, \\ \alpha x, & \text{если } x < 0, \end{cases} \quad \text{где } 0 < \alpha < 1$$

α – параметр, константное малое значение (обычно 0,01) или случайно сгенерированное значение соответственно.

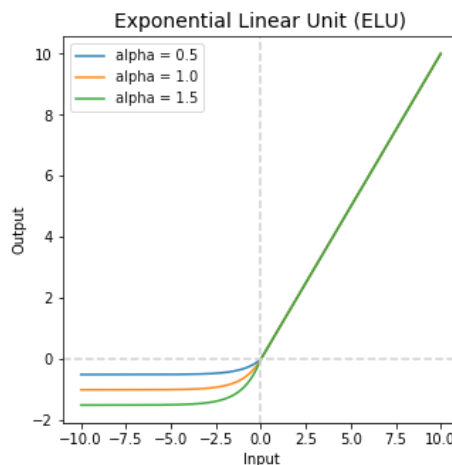


- нелинейная;
- непрерывная, разрыв производной в точке $x = 0$, порядок гладкости C^0 ;
- область значений: $(-\infty; +\infty)$;
- монотонна;
- нелинейна в окрестности нуля.

Один из недостатков функции ReLU – проблема «мертвых» нейронов, у которых в процессе обучения смещение получает излишне высокое отрицательное значение. Из-за искусственного смещения влево входного значения нейрон начинает выдавать значение 0 для любых входов, а при обучении градиент ошибки этого нейрона недостаточен, чтобы вывести его из «мертвого» состояния. Чтобы избавиться от этой проблемы, распространены вариации ReLU, в которых используется линейная функция с другим углом наклона для отрицательных входных значений. С другой стороны, такие функции не настолько производительны при их использовании в процессе обучения.

Exponential Linear Unit (ELU)

$$y(x) = \begin{cases} x, & \text{если } x \geq 0, \\ \alpha(e^x - 1), & \text{если } x < 0 \end{cases}$$



- нелинейная;
- непрерывная, при $\alpha = 1$ непрерывно дифференцируемая с порядком гладкости C^1 , при $\alpha \neq 1$ разрыв производной в $x = 0$ и порядок гладкости C^0 ;
- область значений: $(-\alpha; +\infty)$;
- монотонна при $\alpha \geq 0$;
- линейна в окрестности нуля при $\alpha = 1$.

Еще одна попытка доопределить линейную функцию для отрицательных чисел, в этом случае с использованием экспоненциальной функции, асимптотически стремящейся к некоторой константе $-\alpha$, которая является гиперпараметром. Для $\alpha = 1$ функция непрерывно дифференцируемая и аппроксимирует линейную в начале координат.

Нейронная сеть прямого распространения

Нейронная сеть — это математическая модель, состоящая из множества взаимосвязанных искусственных нейронов, объединенных в слои, которая способна обучаться на данных и выполнять задачи, такие как классификация, регрессия, распознавание образов и др.

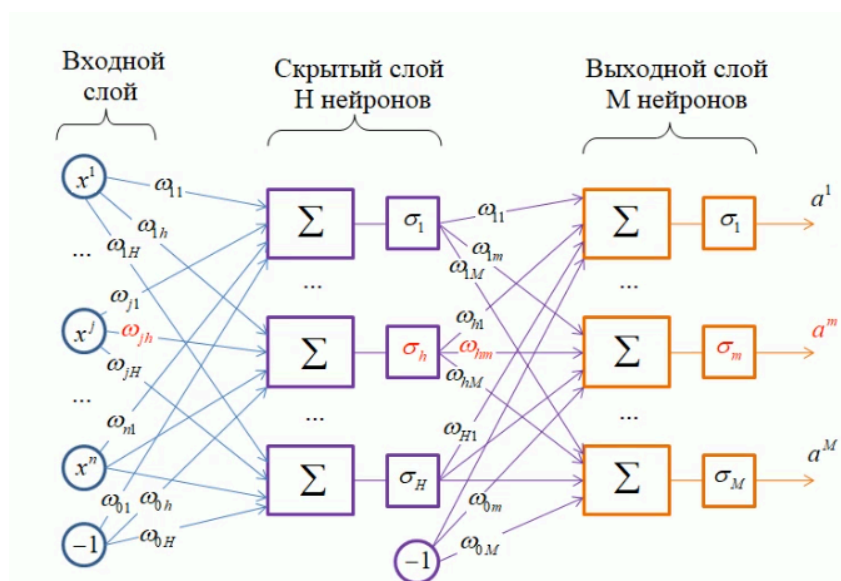
Нейросеть состоит из следующих слоев:

1. Входной слой — принимает исходные данные.
2. Скрытые слои — один или несколько слоев между входом и выходом сети. Именно в этой части происходят основные вычисления и выявления закономерностей во входных данных.
3. Выходной слой — формирует финальный результат сети. На выходе может возвращать вектор значений, например, признаковый вектор.

Нейронная сеть прямого распространения (Feedforward Neural Network, FFNN) — это тип искусственной нейронной сети, в которой информация проходит только в одном направлении, то есть от входного слоя, через скрытые слои и к выходному слою, без обратных связей и циклов.

Полносвязная нейронная сеть (или *полносвязный слой, fully connected layer*) — это нейронная сеть с такой организацией, при которой каждый нейрон одного слоя соединен со всеми нейронами следующего слоя.

Пример полносвязной нейронной сети прямого распространения с одним скрытым слоем:



Многослойный перцептрон (Multi-Layer Perceptron, MLP) — это частный случай *FFNN*, который всегда имеет скрытые слои и полносвязную структуру.

Как было сказано ранее, выходной слой сети может возвращать не только одно значение, но и целый вектор. Поэтому стоит упомянуть функцию активации применяемую к всему выходному слою — **SoftMax**:

$$\sigma_m(v) = \frac{\exp(v^{(m)})}{\sum_{i=1}^M \exp(v^{(i)})}$$

где M — количество нейронов последнего слоя; $v^{(k)}$ — сумма перемножения весов с входными значениями для m -го нейрона выходного слоя.

Такую функцию можно применить на выходе нейронной сети решающей задачу многоклассовой классификации. Тогда значение σ_m будет соответствовать вероятности отнесения к m -ому классу.

Обучение нейронной сети

Как и модели машинного обучения из прошлых лабораторных работ, нейросетевая модель — это вид гипотезы, который задает вид целевой функции, и здесь для обучения также задается функция стоимости и итерационный алгоритм для подстройки целевой функции таким образом, чтобы стоимость, как следствие и ошибка, уменьшалась:

$$Q(W) = \sum_{i=1}^l L_i(W)$$

где l — количество наблюдений в обучающей выборке; W — матрица весов нейронной сети; $L(W)$ — выбранная функция потерь.

Итерационным алгоритмом для оптимизации функции потерь здесь также часто выбирают градиентный спуск. Следуя ему, на каждом шаге алгоритма мы приближаем с помощью антиградиента наши веса к значениям, при которых функция стоимости будет минимальна:

$$w_{p,k,j}^{(t)} = w_{p,k,j}^{(t-1)} - \alpha \frac{\partial L(W^{(t-1)})}{\partial w_{p,k,j}}$$

где $w_{p, k, j}^{(t)}$ — вес связи нейросети между k -ым нейроном слоя p и j -ым нейроном слоя $p+1$ на шаге t , $W^{(t)}$ — матрица весов на шаге t , α — шаг сходимости, или скорость обучения.

Однако, вычисление частной производной функции потерь по весу $w_{k, j}$ сложная задача. И она тем сложнее, чем больше слоев в сети и чем ближе слой k к входному слою. Поэтому специально для нейронных сетей был придуман более упрощенный алгоритм нахождения частных производных — ***Back propagation***.

В ***Back propagation*** вычисление частных производных начинается с последнего слоя. Найденные на последнем слое значения частных производных используются для вычисления частных производных предпоследнего слоя. Таким образом алгоритм идет в обратном направлении по нейронной сети, используя на каждом этапе уже вычисленные частные производные для нахождения новых. Процесс завершается, когда доходит до входного слоя.

Так как в начале алгоритма частная производная на выходном слое — это по сути ошибка, то этот метод также называют ***методом обратного распространения ошибки***.

Тогда корректировка весов нейросети на шаге t относительно одного i -го наблюдения из обучающей выборки выглядит следующим образом:

Прямой проход (Forward Propagation)

Наблюдение i попадает на вход нейросети. Для него на каждом нейроне k со слоя p рассчитываются и запоминаются полученная на входе нейрона сумма $v_{p, k}$ и полученное на выходе нейрона значение $\sigma_{p, k}(v_{p, k})$.

Обратное распространение ошибки (Back Propagation)

Обозначим нейросеть как гипотезу $a(W, x)$, и перепишем функцию потерь следующим образом:

$$L_i(W) = L(a(W, x^{(i)}), y^{(i)})$$

где x и y — наблюдение и его целевое значение из обучающей выборки соответственно.

Начиная с последнего слоя, происходит вычисление частных производных и корректировка весов. Первым делом рассчитывается ошибка для нейронов выходного слоя:

$$\varepsilon_{P,k} = \frac{\partial L(a(W, x), y)}{\partial a}$$

где P — номер выходного слоя, а k — номер нейрона в этом слое.

Следуя названию метода, затем вычисляются ошибки для нейронов с предыдущих слоев по формуле:

$$\varepsilon_{p,k} = \sum_{j=0}^J w_{p,k,j} \cdot \varepsilon_{p+1,j} \cdot \sigma'_{p,k}(v_{p,k})$$

где p — номер слоя, k — номер нейрона на слое p , а j — номер нейрона на слое $p+1$; $w_{p,k,j}$ — вес связи нейросети между k -ым нейроном слоя p и j -ым нейроном слоя $p+1$.

Веса пересчитываются по следующей формуле:

$$w_{p,k,j} = w_{p,k,j} - \alpha \cdot \varepsilon_{p+1,j} \cdot \sigma'_{p+1,j}(v_{p+1,j}) \cdot \sigma_{p,k}(v_{p,k})$$

где p — номер слоя, k — номер нейрона на слое p , а j — номер нейрона на слое $p+1$.

Математический вывод этих формул можно посмотреть в этом [видео](#).

За один шаг t стандартного градиентного спуска совершается l прямых и обратных проходов по сети, то есть происходит пересчет весов относительно каждого наблюдения из обучающей выборки.

Однако это очень трудозатратная задача учитывая, что для нейронных сетей размеры обучающих выборок обычно огромны. Поэтому на практике чаще используют **стохастический градиентный спуск**, в котором на шаге t веса корректируются либо относительно одного наблюдения, либо небольшого их количества, называемого **мини-пакетом (mini-batch)**.

□□□□□Здесь начинается старая методичка□□□□□

Архитектура многослойного персептрона (MLP)

Структура MLP

Многослойный перцептрон (MLP, Multi-Layer Perceptron) — это один из базовых видов искусственных нейронных сетей. Он состоит из нескольких слоев:

- **Входной слой:** получает входные данные (признаки). Количество нейронов = количество признаков.
- **Скрытые слои:** выполняют нелинейное преобразование входных данных.
- **Выходной слой:** формирует предсказания. Количество нейронов зависит от задачи (например, 1 нейрон для бинарной классификации).

Формальное описание нейрона

Каждый искусственный нейрон выполняет следующие вычисления:

1. Линейное преобразование входных данных

$$z = \sum_{i=1}^n w_i x_i + b$$

где:

- x_i — входные значения,
- w_i — веса,
- b — смещение (bias).

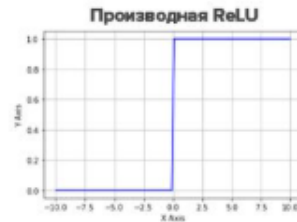
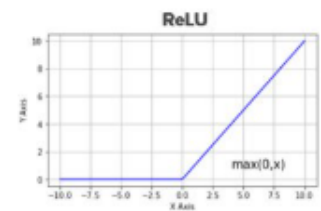
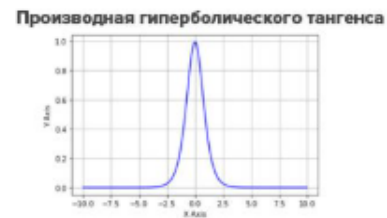
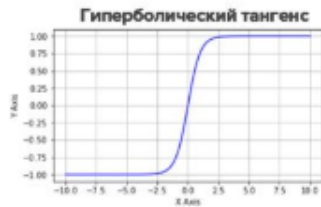
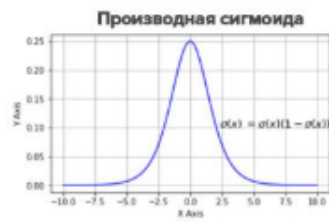
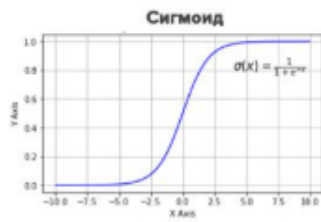
2. Активационная функция

$$a = f(z)$$

где $f(z)$ — функция активации, которая добавляет нелинейность.

Функции активации

Функции активации преобразуют выходное значение нейрона, добавляя нелинейность:



Обучение MLP

Прямой проход (Forward Propagation)

Во время прямого прохода (forward pass) вычисляются выходы каждого слоя:

1. Линейное преобразование:

$$z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}$$

2. Активация:

$$a^{(l)} = f(z^{(l)})$$

3. Повторение для всех слоев.

Функция ошибки (Loss function)

Функция ошибки измеряет, насколько предсказания модели отклоняются от истинных значений.

- Для классификации (кросс-энтропия):

$$L = - \sum y_i \log(\hat{y}_i)$$

- Для регрессии (MSE - среднеквадратичная ошибка):

$$L = \frac{1}{n} \sum (y_i - \hat{y}_i)^2$$

Обратное распространение ошибки (Backpropagation)

Обратное распространение ошибки (backpropagation) используется для нахождения градиентов и обновления весов.

1. Вычисление градиента ошибки по выходу сети.
2. Вычисление градиентов весов с использованием **цепного правила** производных.
3. Обновление весов с использованием градиентного спуска:

$$W = W - \alpha \frac{\partial L}{\partial W}$$

где α — скорость обучения.

Оптимизаторы

Оптимизаторы улучшают процесс градиентного спуска.

Оптимизатор	Описание
SGD	Стохастический градиентный спуск
Adam	Комбинирует RMSProp и Momentum
RMSProp	Использует адаптивную скорость обучения

Гиперпараметры MLP

Многослойный перцептрон имеет множество параметров, влияющих на его работу.

Гиперпараметр	Описание
Количество слоев	Чем больше слоев, тем сложнее модель
Число нейронов в слоях	Больше нейронов → выше мощность модели
Функция активации	Определяет нелинейность модели
Скорость обучения	Маленькая — долгое обучение, большая — нестабильность
Размер мини-пакета	Влияет на сходимость градиентного спуска
Регуляризация (Dropout, L2)	Предотвращает переобучение

Описание эксперимента

Шаги выполнения лабораторной работы:

1. Загрузка данных.
2. Предобработка данных.
3. Создание и обучение MLP.
4. Оценка качества модели.
5. Сравнение с классическими алгоритмами.

Практическая часть (Пример кода на TensorFlow/Keras)

1. Импорт библиотек и загрузка данных

```
import numpy as np
import tensorflow as tf
from tensorflow import keras
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score
from sklearn.datasets import make_moons
import matplotlib.pyplot as plt

# Генерируем синтетический набор данных (пример - две луны)
X, y = make_moons(n_samples=1000, noise=0.2, random_state=42)

# Разделяем на тренировочную и тестовую выборки
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
                                                    random_state=42)

# Нормализация данных
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

2. Создание многослойного перцептрона (MLP)

```
model = keras.Sequential([
    keras.layers.Dense(16, activation='relu', input_shape=(2,)), # Скрытый слой
    keras.layers.Dense(8, activation='relu'), # Скрытый
    keras.layers.Dense(1, activation='sigmoid') # Выходной
])

# Компиляция модели
model.compile(optimizer='adam', loss='binary_crossentropy',
              metrics=['accuracy'])

# Обучение модели
history = model.fit(X_train, y_train, epochs=50, batch_size=16,
                    validation_data=(X_test, y_test))
```

3. Оценка модели

```
# Оцениваем качество на тестовой выборке

y_pred = (model.predict(X_test) > 0.5).astype("int32")

print("Accuracy:", accuracy_score(y_test, y_pred))


# График изменения функции ошибки

plt.plot(history.history['loss'], label='Training Loss')

plt.plot(history.history['val_loss'], label='Validation Loss')

plt.xlabel('Epochs')

plt.ylabel('Loss')

plt.legend()

plt.show()
```

4. Сравнение с классическими алгоритмами

```
from sklearn.ensemble import RandomForestClassifier

from sklearn.linear_model import LogisticRegression


# Логистическая регрессия

lr = LogisticRegression()

lr.fit(X_train, y_train)

y_pred_lr = lr.predict(X_test)

print("Logistic Regression Accuracy:", accuracy_score(y_test, y_pred_lr))


# Случайный лес

rf = RandomForestClassifier(n_estimators=100)

rf.fit(X_train, y_train)

y_pred_rf = rf.predict(X_test)

print("Random Forest Accuracy:", accuracy_score(y_test, y_pred_rf))
```

Задание

“Выборы президента”

Цель работы:

Разработать и обучить нейросеть, которая предсказывает победу на выборах на основе данных об ответах избирателей на 12 вопросов.

Задачи:

1. Подготовить данные для обучения модели.
2. Создать нейросеть с одним скрытым слоем и функцией активации `logsig`.
3. Обучить модель на предоставленных данных.
4. Проверить её точность на тестовых примерах.
5. Визуализировать результаты предсказаний.

Данные:

Входные данные (`dataIn.txt`): матрица ($12 \times N$), где N — количество примеров. Каждый столбец — это 12 ответов одного избирателя.

Выходные данные (`dataOut.txt`): матрица ($2 \times N$), где каждая колонка `[1 0]` означает победу правящей партии, а `[0 1]` — победу оппозиции.

Данные:

```
# Генерация тестовых данных

np.random.seed(42)

X = np.random.randint(0, 2, size=(100, 12)) # 100 примеров, 12 бинарных признаков
Y = np.random.randint(0, 2, size=(100, 2)) # 100 примеров, 2 класса (one-hot encoding)

# Сохранение данных в файлы

np.savetxt('dataIn.txt', X, fmt='%d')
np.savetxt('dataOut.txt', Y, fmt='%d')
```